Sécurité des applications et cycle de développement sécurisé

Informatique — Licence 3 (L3)

Cours INF-L3-13

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours
Intégrer la sécurité dès la conception d'une application : activités de sécurité à chaque phase du cycle de vie, revue de code orientée sécurité, gestion rigoureuse des secrets, journalisation utile sans fuites, correctifs priorisés et divulgation responsable.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Le cycle de développement sécurisé : principes et étapes
• Leçon 2 — Revue de code orientée sécurité
• Leçon 3 — Gestion sécurisée des secrets et des configurations
• Leçon 4 — Journalisation et détection des intrusions applicatives
• Leçon 5 — Correctifs et gestion des vulnérabilités connues
• Leçon 6 — Programmes de bug bounty et divulgation responsable
• Leçon 7 — Projet : durcir la sécurité d'une application existante
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

La plupart des failles qui font la une des journaux ne naissent pas dans un centre d'opérations sophistiqué : elles naissent dans du code ordinaire, écrit par des développeurs pressés qui ignoraient qu'une requête construite par concaténation, un mot de passe commité par erreur ou un journal trop bavard suffisent à ouvrir une brèche. Corriger une faille à la conception coûte quelques minutes ; la corriger après un incident coûte des semaines, de l'argent et parfois la confiance des utilisateurs. Ce cours part de ce constat simple : la sécurité d'une application se joue d'abord au moment où on l'écrit.

Tu vas y apprendre une méthode complète et défensive : une activité de sécurité à chaque phase du cycle de développement, une grille de revue de code que tu sauras appliquer à n'importe quel module, une discipline stricte pour les secrets — avec la procédure exacte à suivre le jour où un secret finit malgré tout dans l'historique Git —, une journalisation qui aide à détecter les incidents sans jamais exposer de mots de passe ni de jetons, une priorisation raisonnée des correctifs, et enfin le cadre légal et contractuel de la divulgation responsable et des programmes de bug bounty. Le fil rouge du cours est l'application BiblioIAM, notre bibliothèque numérique, que nous durcirons leçon après leçon.

Ce cours prolonge la cybersécurité défensive de L3 (INF-L3-01), la conception logicielle (INF-L3-04) et les pipelines de livraison (INF-L3-12). Il suppose que tu sais déjà écrire une application web simple, utiliser Git et interroger une base de données. Sa règle absolue : nous sécurisons notre propre code et nos propres systèmes. Tester le

système d'autrui sans autorisation écrite n'est ni un exercice, ni un jeu : c'est un délit, et ce cours ne t'y préparera jamais. Les programmes de bug bounty y sont présentés pour ce qu'ils sont : des contrats qui définissent un périmètre, des règles et des récompenses — rien de plus, rien de moins.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Décrire les étapes d'un cycle de développement sécurisé et l'activité de sécurité associée à chaque phase.
- Mener une revue de code orientée sécurité avec une grille : entrées, requêtes, authentification, autorisations, erreurs.
- Sortir tous les secrets du code source : variables d'environnement, fichiers ignorés, coffres à secrets.
- Appliquer la procédure complète lorsqu'un secret a été commité : rotation immédiate, purge de l'historique, audit des accès.
- Journaliser les événements de sécurité utiles en masquant systématiquement mots de passe, jetons et données sensibles.
- Prioriser et appliquer les correctifs de vulnérabilités connues selon la criticité et l'exposition réelle.
- Expliquer le cadre légal et contractuel des programmes de bug bounty et rédiger un rapport de divulgation responsable.

Leçon 1 — Le cycle de développement sécurisé : principes et étapes

Un cycle de développement sécurisé (on parle souvent de SDLC sécurisé, pour Secure Development LifeCycle) consiste à ajouter une activité de sécurité à chaque phase de la vie d'un logiciel, au lieu de « faire la sécurité » à la fin, juste avant la mise en production. Ce déplacement de l'effort vers l'amont porte un nom : le shift left, « déplacer vers la gauche », la gauche étant le début du calendrier du projet. L'idée n'est pas de transformer chaque développeur en expert en sécurité, mais de faire en sorte que les erreurs les plus fréquentes et les plus graves soient empêchées ou détectées au moment où elles coûtent le moins cher à corriger.

Concrètement, chaque phase a son activité. Aux exigences, on écrit aussi des exigences de sécurité (« les mots de passe seront stockés hachés », « un membre ne voit que ses propres emprunts ») et l'on imagine les abus possibles. À la conception, on dessine un modèle de menaces simple : quelles données ont de la valeur, qui pourrait les convoiter, par où passerait-il ? Pendant le codage, on applique des règles écrites (requêtes paramétrées, validation des entrées), on relit le code entre pairs et l'on branche des analyses automatiques. Aux tests, on ajoute des cas d'abus : que se passe-t-il si j'envoie un texte malformé, si je demande la fiche d'un autre utilisateur ? Au déploiement, on durcit la configuration ; en exploitation, on lit les journaux et l'on suit les correctifs. Des référentiels publics structurent cette démarche : le SDL de Microsoft, le modèle SAMM de l'OWASP et le cadre SSDF du NIST les décrivent chacun à leur manière.

Un cycle sécurisé doit rester proportionné au projet. Une application de stage n'a pas besoin de la même machinerie qu'une plateforme bancaire, mais aucune n'est dispensée des fondamentaux : secrets hors du code, requêtes paramétrées, correctifs suivis. Deux habitudes font la différence dans une équipe : d'abord, un « référent sécurité » tournant qui garde les sujets de sécurité visibles dans les réunions ; ensuite, une définition de « terminé » qui inclut la sécurité — une fonctionnalité n'est pas finie tant que ses entrées ne sont pas validées et son code relu. La sécurité cesse alors d'être un contrôle subi pour devenir un critère de qualité comme un autre.

L'essentiel
Le SDLC sécurisé ajoute une activité de sécurité à chaque phase (exigences, conception, code, tests, déploiement, exploitation) au lieu d'un contrôle final. Shift left : détecter tôt coûte moins cher. L'effort est proportionné au projet, mais les fondamentaux ne sont jamais optionnels.

Exemple — Carte des activités de sécurité par phase, pour l'application BiblioIAM

```
Phase Activité de sécurité Livrable produit
------------- ------------------------------------- ------------------------
Exigences Lister les abus possibles en équipe Exigences de sécurité
Conception Modèle de menaces simple sur schéma Décisions validées
Code Revue orientée sécurité + analyse Rapport de revue signé
automatique du code
Tests Cas d'abus : entrées malformées, Résultats corrigés
accès aux données d'autrui
Déploiement Configuration durcie, secrets hors Check-list de mise en
du code, comptes par défaut changés production
Exploitation Journaux relus chaque semaine, Registre des correctifs
correctifs de sécurité suivis à jour
```

À retenir

- Shift left : une faille trouvée à la conception coûte des minutes ; trouvée après un incident, des semaines.
- À chaque phase correspond une activité de sécurité précise et un livrable vérifiable.
- La définition de « terminé » d'une fonctionnalité inclut ses critères de sécurité, sinon elle n'est pas terminée.

Exercice

Une équipe développe une application de prise de rendez-vous pour un cabinet médical. Pour chacune des situations suivantes, indique à quelle phase du cycle sécurisé elle appartient et ce qui cloche : (1) la veille de la mise en production, quelqu'un demande « au fait, les mots de passe sont stockés comment ? » ; (2) dès la première réunion, l'équipe écrit : « un patient ne peut voir que ses propres rendez-vous » ; (3) un test vérifie que le formulaire rejette un numéro de téléphone contenant des lettres ; (4) trois mois après le lancement, personne ne sait quelles bibliothèques l'application utilise ni si elles ont des failles connues.

✔ Corrigé
Situation (1) : phase de déploiement — le défaut est d'avoir attendu la fin pour poser une question de conception ; le stockage des mots de passe (hachage adapté) se décide à la conception et se vérifie à la revue de code. Situation (2) : phase d'exigences — c'est exactement le bon réflexe, une exigence de sécurité claire et testable, écrite dès le départ. Situation (3) : phase de tests — un cas d'abus bien formé : il vérifie la validation des entrées, pas seulement le fonctionnement nominal. Situation (4) : phase d'exploitation — il manque l'inventaire des dépendances et le suivi des correctifs (leçon 5) ; sans inventaire, impossible de savoir si l'application est concernée par une vulnérabilité publiée. Le fil conducteur : chaque situation aurait dû être traitée dans SA phase, ni avant ni après.

Leçon 2 — Revue de code orientée sécurité

La revue de code orientée sécurité est une relecture du code source par un pair, avec un objectif précis : y traquer les défauts qui peuvent devenir des failles. Elle ne remplace ni les tests ni les analyses automatiques — elle les complète, car un humain comprend l'intention du code et repère les erreurs de logique que les outils ne voient pas : un contrôle d'autorisation oublié sur une seule route, une comparaison de jeton mal écrite, un message d'erreur qui en dit trop. Dans la plupart des équipes, elle prend la forme d'une relecture de demande de fusion (pull request), avant que le code n'atteigne la branche principale.

Pour être efficace, la revue suit une grille et un chemin. Le chemin, c'est celui des données : on repère d'où viennent les entrées (formulaires, paramètres d'URL, fichiers reçus, en-têtes) et on les suit jusqu'aux endroits sensibles — la base de données, l'affichage HTML, les commandes système, les fichiers. À chaque passage, on pose les mêmes questions : l'entrée est-elle validée ? La requête est-elle paramétrée ? La sortie est-elle échappée ? L'utilisateur a-t-il le droit de faire cette action précise, et le code le vérifie-t-il côté serveur ? Les erreurs révèlent-elles des détails internes ? Un secret traîne-t-il dans le code ou les commentaires ? Cette discipline transforme une relecture impressionniste en contrôle systématique.

La posture compte autant que la technique. Une revue de sécurité n'est ni un procès ni un concours : on critique le code, jamais la personne, et chaque remarque explique le risque puis propose la correction. Tout ce qui est mécanique — style, motifs dangereux connus — doit être délégué aux analyseurs automatiques (analyse statique, détecteurs de secrets) pour que le temps humain serve aux raisonnements. Enfin, on concentre l'effort : authentification, autorisations, paiements, données personnelles et accès à la base méritent une relecture lente et complète ; un changement d'intitulé de bouton, non. C'est cette priorisation qui rend la revue soutenable semaine après semaine.

L'essentiel
Relire le code en suivant le chemin des données, de l'entrée jusqu'aux zones sensibles, avec une grille de questions fixes : validation, paramétrage, échappement, autorisation côté serveur, messages d'erreur sobres, aucun secret. L'humain juge la logique ; les outils font le mécanique.

Exemple — Le même code, vulnérable puis corrigé : recherche de livres dans BiblioIAM (Python)

```
# ============ VERSION VULNÉRABLE — étiquetée, À NE JAMAIS UTILISER ============
def chercher_vulnerable(connexion, texte):
# La saisie est collée dans la requête : un texte habilement formé
# peut modifier le sens de la requête (injection SQL).
requete = "SELECT titre FROM livres WHERE titre LIKE '%" + texte + "%'"
return connexion.execute(requete).fetchall()

def page_vulnerable(texte):
# La saisie est renvoyée telle quelle dans la page (script injecté).
return "<h1>Résultats pour " + texte + "</h1>"

# ============ VERSION CORRIGÉE ============
import html

def chercher(connexion, texte):
# La requête est paramétrée : le texte reste une DONNÉE,
# il ne peut plus devenir du code SQL.
requete = "SELECT titre FROM livres WHERE titre LIKE ?"
return connexion.execute(requete, ("%" + texte + "%",)).fetchall()

def page_resultats(texte):
# L'échappement neutralise les caractères spéciaux du HTML.
return "<h1>Résultats pour " + html.escape(texte) + "</h1>"

def detail_livre(connexion, identifiant):
# Validation d'entrée : un identifiant doit être un entier.
if not str(identifiant).isdigit():
raise ValueError("Identifiant invalide")
requete = "SELECT titre FROM livres WHERE id = ?"
return connexion.execute(requete, (int(identifiant),)).fetchone()
```

À retenir

- On suit les données : chaque entrée utilisateur est suivie jusqu'à la base, à l'écran ou au fichier qu'elle atteint.
- Requête paramétrée, sortie échappée, entrée validée : le trio qui ferme la majorité des failles du code web.
- L'autorisation se vérifie côté serveur, sur chaque action — jamais seulement en cachant un bouton.

Exercice

Voici trois extraits décrits d'un module de commentaires d'un blog. Pour chacun, dis si la revue doit le signaler, pourquoi, et quelle correction tu proposes. Extrait A : le

commentaire est enregistré puis affiché aux visiteurs exactement comme il a été saisi. Extrait B : pour supprimer un commentaire, le code vérifie seulement que l'utilisateur est connecté, pas qu'il est l'auteur du commentaire ou modérateur. Extrait C : quand l'enregistrement échoue, la page affiche le message technique complet de la base de données, avec le nom des tables.

✔ Corrigé
Extrait A : à signaler — affichage sans échappement : un commentaire contenant du code de script serait exécuté dans le navigateur des visiteurs (injection de script). Correction : échapper systématiquement à l'affichage (ou utiliser le moteur de gabarits qui échappe par défaut) et ne jamais stocker « pré-échappé ». Extrait B : à signaler — c'est un défaut d'autorisation : être connecté ne donne pas le droit de supprimer le commentaire d'autrui. Correction : vérifier côté serveur, pour chaque action, que l'utilisateur est l'auteur ou possède le rôle modérateur. Extrait C : à signaler — fuite d'informations : les messages techniques aident un attaquant à cartographier la base. Correction : journaliser le détail côté serveur (leçon 4) et n'afficher à l'utilisateur qu'un message générique du type « une erreur est survenue ». Trois familles différentes — injection, autorisation, exposition d'informations — : la grille de revue sert précisément à ne pas passer à côté de l'une d'elles.

Leçon 3 — Gestion sécurisée des secrets et des configurations

Un secret est toute information qui donne un pouvoir : mot de passe de la base de données, clé d'API de paiement, jeton d'accès, clé de chiffrement, secret de session. Le code source est le pire endroit où le ranger, pour trois raisons. Le code se copie : chaque développeur, chaque sauvegarde, chaque service d'intégration en détient un exemplaire. Le code se versionne : un secret commité une fois reste dans l'historique Git même si on le retire ensuite — et l'historique se clone. Le code se partage : capture d'écran, extrait collé dans un forum, dépôt devenu public par erreur. Des programmes automatiques scrutent en permanence les dépôts publics à la recherche de clés fraîchement publiées : un secret exposé peut être exploité en quelques minutes.

La bonne pratique sépare le code de la configuration. Le code lit ses secrets dans des variables d'environnement fournies par la machine qui l'exécute ; en développement, ces variables vivent dans un fichier local (.env) que Git ignore grâce au .gitignore, et le dépôt ne contient qu'un modèle (.env.exemple) qui liste les noms des variables sans aucune vraie valeur. Chaque environnement (développement, test, production) possède ses propres secrets, et chaque secret ne donne que les droits nécessaires : la clé qui sert à afficher des statistiques n'a pas à pouvoir modifier la base. Pour les systèmes plus importants, les coffres à secrets dédiés — proposés par les hébergeurs cloud ou par des logiciels spécialisés — ajoutent le chiffrement, le contrôle d'accès nominatif, la journalisation des lectures et la rotation automatique.

Reste l'accident, qui arrivera un jour ou l'autre : un secret commité puis poussé. La procédure est immuable et l'ordre compte. D'abord, et avant tout le reste : révoquer le secret chez le fournisseur et en générer un nouveau — le secret exposé doit être considéré comme public pour toujours, le supprimer du code ne le « dés-expose » pas.

Ensuite, vérifier les journaux du service concerné pour repérer un usage inconnu pendant la fenêtre d'exposition. Puis seulement, nettoyer : purger l'historique du dépôt (outil de réécriture d'historique, ou dépôt recréé proprement). Enfin, empêcher la récidive : .gitignore complet, détecteur de secrets en pre-commit et dans le pipeline, et incident consigné au registre. Nous appliquons cette procédure en travaux dirigés.

L'essentiel
Un secret ne vit jamais dans le code : variables d'environnement, fichier local ignoré par Git, modèle sans valeurs, coffre dédié pour les gros systèmes. Si un secret a été commité, l'ordre est : 1) le révoquer et le régénérer, 2) auditer les accès, 3) purger l'historique, 4) prévenir la récidive.

Exemple — Secrets hors du code : .gitignore, modèle, lecture d'environnement et procédure d'incident

```
# ---------- .gitignore (à commiter) ----------
.env
.env.*
!.env.exemple
*.pem
*.key

# ---------- .env.exemple (à commiter : des NOMS, jamais de vraies valeurs)
BIBLIO_DB_URL=sqlite:///biblio.db # exemple de valeur locale
BIBLIO_CLE_API_PAIEMENT=exemple-a-remplacer
BIBLIO_SECRET_SESSION=exemple-a-remplacer

# ---------- code Python : lire les secrets dans l'environnement ----------
import os

# Lève une erreur claire si la variable manque : on échoue tôt,
# jamais on ne retombe sur une valeur par défaut codée en dur.
CLE_PAIEMENT = os.environ["BIBLIO_CLE_API_PAIEMENT"]
SECRET_SESSION = os.environ["BIBLIO_SECRET_SESSION"]

# ---------- SI UN SECRET A ÉTÉ COMMITÉ : procédure, dans l'ordre ----------
# 1. RÉVOQUER le secret chez le fournisseur et en générer un nouveau.
# (Un secret exposé est public pour toujours, même retiré du code.)
# 2. AUDITER les journaux du service : un usage inconnu pendant
# la période d'exposition ?
# 3. PURGER l'historique Git (réécriture d'historique ou dépôt recréé),
# puis forcer la resynchronisation des clones de l'équipe.
# 4. PRÉVENIR : .gitignore, détecteur de secrets en pre-commit et
# dans le pipeline, incident inscrit au registre de sécurité.
```

À retenir

• Code et configuration sont séparés : le code lit des variables d'environnement, il ne contient aucun secret.

• Le dépôt versionne un modèle (.env.exemple) avec des noms et des valeurs fictives étiquetées « exemple » — jamais les vraies.

• Secret commité = secret mort : rotation d'abord, audit ensuite, purge de l'historique après, prévention enfin.

Exercice

Dans le dépôt d'une association, un étudiant trouve : (a) un fichier config.py contenant le vrai mot de passe de la base de production, commité il y a deux mois ; (b) un fichier .env.exemple qui contient, par commodité, la vraie clé d'API de test, « parce que c'est seulement le test » ; (c) un mot de passe de démonstration « admin123 » codé en dur pour le compte de démonstration, désactivé en production par un réglage. Pour chaque cas, dis s'il y a un problème, sa gravité, et le geste correct.

✔ Corrigé

Cas (a) : problème grave et ancien — le mot de passe a vécu deux mois dans un historique clonable. Procédure complète : changer immédiatement le mot de passe de la base (rotation), vérifier les journaux de la base pour des connexions inconnues sur la période, purger l'historique, mettre le secret en variable d'environnement. Le fait qu'il soit « caché » dans un vieux commit ne le protège pas. Cas (b) : problème réel — une clé « de test » reste une clé valide du fournisseur ; elle peut ouvrir des droits inattendus, servir de marchepied, ou être confondue un jour avec celle de production. On la révoque, on en génère une autre hors dépôt, et le modèle ne garde que le nom de la variable. Cas (c) : problème de conception — un secret codé en dur protégé « par un réglage » redevient actif dès que le réglage change ou que le code est réutilisé ailleurs. Correction : le compte de démonstration doit être créé par la configuration (secret lu dans l'environnement), ou simplement absent du code livré. Point commun des trois cas : un secret ne se protège pas par la discrétion, mais par l'absence du code et la rotation en cas d'exposition.

Leçon 4 — Journalisation et détection des intrusions applicatives

Quand un incident survient, la première question est toujours : que s'est-il passé, et quand ? Sans journaux (logs), il n'y a pas de réponse — seulement des suppositions. Bien journaliser, c'est enregistrer les événements qui comptent pour la sécurité : connexions réussies et échouées, refus d'autorisation (« un utilisateur a demandé une ressource interdite »), actions sensibles (changement de mot de passe, suppression, modification de rôles), erreurs de validation répétées, actions d'administration. Chaque entrée porte au minimum un horodatage fiable, l'acteur (identifiant du compte, pas son nom complet si ce n'est pas nécessaire), l'action, la ressource concernée, le résultat et un identifiant de corrélation qui relie les entrées d'une même requête. Des journaux structurés (format JSON, un champ par information) se relisent par des outils, pas à l'œil sur des milliers de lignes.

L'autre moitié du sujet est ce qu'il ne faut JAMAIS journaliser. Un journal est un fichier comme un autre : il se sauvegarde, se copie, se consulte — parfois par des personnes ou des prestataires qui n'auraient jamais dû voir un mot de passe. On n'y écrit donc ni mots de passe (même « pour vérifier »), ni jetons de session ou d'API complets, ni numéros de carte complets, ni secrets d'aucune sorte, ni contenus privés des

utilisateurs sans nécessité. Quand une information sensible est indispensable au diagnostic, on la masque : ne conserver que les derniers caractères d'un jeton, remplacer le mot de passe par un marqueur fixe, hacher l'identifiant plutôt que l'afficher en clair. La règle de décision est simple : si la fuite du journal lui-même devenait un incident, c'est qu'il contenait trop de choses.

Des journaux bien tenus deviennent un outil de détection. Centralisés (l'application envoie ses journaux vers un espace dédié, en ajout seul, pour qu'un intrus qui compromettrait l'application ne puisse pas effacer ses traces), ils alimentent des règles d'alerte simples et efficaces : un grand nombre d'échecs de connexion depuis une même adresse en quelques minutes, une rafale de refus d'autorisation sur un même compte, une action d'administration hors des horaires habituels, une explosion d'erreurs 500 après un déploiement. Chaque alerte doit arriver quelque part où un humain la verra, avec assez de contexte pour décider. On règle aussi la durée de conservation et l'accès aux journaux : ce sont des données personnelles potentielles, leur collecte obéit donc à une finalité et à une durée déclarées, pas à « on verra bien ».

L'essentiel
Journaliser les événements de sécurité (connexions, refus, actions sensibles) en format structuré, horodaté et corrélé. Ne jamais écrire mots de passe, jetons complets ni secrets : masquer ou tronquer. Centraliser en ajout seul et brancher quelques règles d'alerte lisibles par un humain.

Exemple — Journaux structurés avec masquage, et trois règles d'alerte (Python et exemples de lignes)

```
import json, logging, time

def masquer(jeton):
# Ne garder que la fin : utile au diagnostic, inutilisable pour entrer.
return "***" + jeton[-4:] if jeton and len(jeton) > 4 else "***"

def journal_securite(evenement, acteur, ressource, resultat, jeton=None):
ligne = {
"horodatage": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
"evenement": evenement, # connexion, refus_acces, admin...
"acteur": acteur, # identifiant du compte uniquement
"ressource": ressource,
"resultat": resultat, # succes / echec / refuse
}
if jeton:
ligne["jeton"] = masquer(jeton) # jamais le jeton complet
logging.getLogger("securite").info(json.dumps(ligne))

# Lignes produites (exemples) :
# {"horodatage": "2026-03-04T09:12:44Z", "evenement": "connexion",
# "acteur": "u1042", "ressource": "/connexion", "resultat": "echec"}
# {"horodatage": "2026-03-04T09:13:02Z", "evenement": "refus_acces",
# "acteur": "u1042", "ressource": "/admin/utilisateurs", "resultat": "refuse"}
# Le mot de passe saisi n'apparaît NULLE PART : ni en échec, ni en succès.

# ---- Règles d'alerte branchées sur ces journaux ----
# A1 : 20 échecs de connexion depuis une même adresse en 5 minutes
# -> possible devinette automatisée de mots de passe : alerter.
# A2 : 10 refus d'accès pour un même compte en 10 minutes
# -> possible exploration de droits : alerter.
# A3 : action "admin" entre 23 h et 5 h alors que l'équipe ne travaille
# que de jour -> vérifier immédiatement avec la personne.
```

À retenir

• On journalise les événements de sécurité : connexions, refus d'autorisation, actions sensibles et d'administration.

• Jamais de mots de passe ni de jetons complets dans les journaux : masquage et troncature systématiques.

• Des journaux centralisés en ajout seul + quelques règles d'alerte simples valent mieux qu'un océan de lignes jamais relues.

Exercice

Voici quatre lignes qu'un développeur propose d'ajouter aux journaux d'une application de paiement scolaire. Pour chacune, dis si elle est acceptable telle quelle, et sinon réécris-la correctement. (1) « Connexion échouée pour awa@exemple.ml, mot de passe saisi : Soleil2026 » ; (2) « Paiement accepté, carte 4998 1234 5678 9012, montant 25 000 » ; (3) « Jeton d'API du partenaire appelé : sk-exemple-9f3K… » (jeton fictif, noté en entier dans la ligne proposée) ; (4) « L'utilisateur u207 a tenté d'ouvrir la page /admin/frais : accès refusé ».

✔ Corrigé
Ligne (1) : inacceptable — un mot de passe, même faux ou mal saisi, ne s'écrit jamais ; de plus, les gens se trompent souvent d'un caractère, le journal révélerait presque le vrai. Réécriture : événement « connexion », acteur = identifiant du compte, résultat = « échec », sans le mot de passe. Ligne (2) : inacceptable — un numéro de carte complet n'a rien à faire dans un journal ; on conserve la référence de transaction et, si le diagnostic l'exige, les quatre derniers chiffres seulement. Ligne (3) : inacceptable — un jeton complet, même d'un partenaire, est un secret ; réécriture avec masquage (« *** » + 4 derniers caractères) ou simple mention « appel partenaire réussi/échoué » sans jeton. Ligne (4) : acceptable et même exemplaire — acteur, ressource, décision : c'est précisément l'événement qui nourrit la règle d'alerte A2 de la leçon. Moralité : la question n'est pas « est-ce utile ? » mais « si ce journal fuitait demain, qu'apprendrait-on ? ».

Leçon 5 — Correctifs et gestion des vulnérabilités connues

Une grande partie des compromissions réussies n'utilise pas des failles inconnues et mystérieuses, mais des vulnérabilités publiques, documentées, pour lesquelles un correctif existe depuis des semaines — parfois des années. La raison est prosaïque : une application moderne repose sur des dizaines de bibliothèques, elles-mêmes appuyées sur d'autres (les dépendances transitives), et personne ne peut suivre tout cela à la main. La première étape de la gestion des vulnérabilités connues est donc un inventaire vivant : la liste exacte des composants et de leurs versions, y compris transitifs — c'est ce qu'on appelle un SBOM, inventaire logiciel. Sans lui, lorsqu'une faille grave est annoncée, impossible de répondre à la seule question qui compte : sommes-nous concernés ?

Les vulnérabilités publiques sont cataloguées sous des identifiants CVE, avec des descriptions et, souvent, un score de sévérité CVSS entre 0 et 10 calculé à partir de caractéristiques comme la facilité d'exploitation et l'impact. Ce score aide à trier, mais il ne décide pas seul : une faille « moyenne » sur un composant exposé à Internet et manipulant des données de paiement peut passer avant une faille « critique » dans un outil interne hors ligne. La priorisation combine donc la sévérité, l'exposition réelle du composant dans TON application, l'existence d'exploitations actives connues et la sensibilité des données atteignables. Des outils d'analyse de composition logicielle (SCA) automatisent la surveillance : ils comparent l'inventaire aux bases de vulnérabilités et alertent dès qu'un composant devient fautif.

Corriger est un processus, pas un réflexe. On définit à l'avance des délais cibles selon la criticité — par exemple correctif critique exposé sous quelques jours, élevé sous quelques semaines — puis on suit le circuit complet : appliquer la mise à jour dans un environnement de test, vérifier que l'application fonctionne toujours (un correctif qui casse tout sera refusé la prochaine fois), déployer, et contrôler que la vulnérabilité a bien disparu. Quand la correction immédiate est impossible (composant abandonné, mise à jour incompatible), on consigne une exception datée avec une mesure de réduction du risque : composant isolé, fonction désactivée, surveillance renforcée — et

une date de réexamen. Ce qui n'est jamais acceptable, c'est l'oubli silencieux : une faille connue, non corrigée et non documentée, devient une faute de gestion autant qu'un défaut technique.

L'essentiel
Gérer les vulnérabilités connues = un inventaire vivant des dépendances (SBOM), une surveillance des CVE, une priorisation qui croise sévérité CVSS, exposition réelle et sensibilité des données, puis un circuit de correction testé avec des délais cibles. L'exception impossible à corriger se documente et se date, ne s'oublie jamais.

Exemple — Registre de tri : quatre vulnérabilités (toutes fictives, pour l'exercice) et leurs décisions

```
# Registre de suivi — BiblioIAM (composants et CVE INVENTÉS pour ce cours)
#
# ID Composant Score Exposition réelle Décision
# --------- -------------------- ------ ----------------------- ---------------
# EXEMPLE-1 biblio-pdf 2.4.1 9,1 Génère les attestations Corriger sous
# Faille d'exécution téléchargeables par 72 h, tester,
# à distance les membres (Internet) déployer
# EXEMPLE-2 biblio-images 1.8.0 7,5 Redimensionne les photos Corriger sous
# Déni de service envoyées par les 7 jours,
# par image piégée membres (Internet) limiter la
# taille d'envoi
# EXEMPLE-3 outil-rapports 0.9 8,2 Génère un rapport Exception
# Faille critique interne, hors ligne, datée : isolé
# (composant abandonné) accès équipe seulement du réseau,
# remplacement
# planifié
# EXEMPLE-4 biblio-core 3.1.0 4,3 Fonction interne peu Corriger à la
# Fuite d'info mineure utilisée, non exposée prochaine
# version
#
# Le score ne trie pas tout seul : EXEMPLE-3 (8,2) passe APRÈS
# EXEMPLE-2 (7,5) parce que l'exposition réelle est plus faible.
# Chaque ligne a un responsable et une date de réexamen.
```

À retenir

- Sans inventaire des dépendances (SBOM), impossible de savoir si une vulnérabilité publiée te concerne.
- La priorité croise le score CVSS avec l'exposition réelle et la sensibilité des données — jamais le score seul.
- Un correctif suit un circuit : test, déploiement, vérification ; une exception se documente avec une date, ne s'oublie pas.

Exercice

Le matin même, trois alertes arrivent pour une application de gestion d'école exposée sur Internet : (A) faille critique (9,8) dans la bibliothèque de génération de PDF utilisée pour les bulletins téléchargeables par les parents ; (B) faille élevée (7,8) dans un composant d'envoi d'e-mails, alors qu'une exploitation active est signalée par l'éditeur

; (C) faille critique (9,0) dans un outil d'administration installé sur un poste interne, jamais exposé, dont l'éditeur a cessé toute maintenance. Propose un ordre de traitement justifié et, pour chaque cas, la première action concrète de la journée.

✔ Corrigé
Ordre proposé : A d'abord, B ensuite de très près, C en risque maîtrisé documenté. Cas A : criticité maximale ET exposition directe aux utilisateurs sur une fonction très visible : première action, vérifier si une version corrigée existe et la tester en priorité absolue ; si le délai dépasse la journée, réduire l'exposition (par exemple suspendre temporairement le téléchargement ou restreindre son accès) plutôt que d'attendre en silence. Cas B : score inférieur mais exploitation active connue : l'urgence réelle est comparable ; première action, appliquer la mise à jour de l'éditeur dès sa disponibilité et surveiller les journaux d'envoi pour des volumes anormaux — l'existence d'attaques en cours fait monter ce cas presque au niveau de A. Cas C : critique sur le papier, mais non exposé et sans éditeur : aucun correctif ne viendra ; première action, documenter l'exception datée, vérifier l'isolation du poste (pas d'accès réseau entrant, comptes limités) et inscrire le remplacement de l'outil au planning avec un responsable. Le corrigé illustre la règle de la leçon : sévérité × exposition × activité d'exploitation, pas le score seul.

Leçon 6 — Programmes de bug bounty et divulgation responsable

Il arrive qu'une personne extérieure découvre une faille dans un système qu'elle utilise. La divulgation responsable (ou coordonnée) est le cadre civilisé pour cette situation : le découvreur signale la faille en privé à l'organisation concernée, lui laisse un délai raisonnable pour corriger — une convention fréquente parle de quelques semaines à quelques mois selon la gravité —, et ne publie les détails qu'après correction ou accord. Ce cadre protège tout le monde : les utilisateurs, qui ne sont pas exposés pendant que la faille est ouverte ; l'organisation, qui peut corriger sans panique publique ; et le découvreur, qui agit de façon reconnue et documentée au lieu de semer le doute sur ses intentions.

Un programme de bug bounty formalise cette relation par un contrat public. L'organisation y définit un périmètre précis (quels sites, quelles applications, quelles catégories de failles sont acceptés), des règles (par exemple : ne pas accéder aux données d'autrui au-delà de la démonstration minimale, ne pas dégrader le service, utiliser ses propres comptes de test), une grille de récompenses selon la gravité, et un engagement de non-poursuite (safe harbor) pour les recherches menées dans les règles. Hors de ce cadre, le même geste technique change de nature juridique : tester un système sans autorisation expose à des poursuites pénales — les législations, dont celle du Mali et de la France, punissent l'accès et le maintien non autorisés dans un système informatique. La frontière n'est pas l'intention, c'est l'autorisation écrite et le respect du périmètre.

Côté organisation, tenir un programme ou simplement une adresse de signalement (un fichier security.txt à la racine du site, une page « sécurité » claire) est une marque de maturité : accuser réception vite, tenir le découvreur informé, corriger dans les délais

annoncés, remercier et, le cas échéant, récompenser comme promis. Côté étudiant, la voie d'entrée honnête dans ce monde est de ne participer qu'à des programmes publics explicites, de lire le périmètre en entier avant le moindre test, de documenter proprement ses rapports et d'accepter les refus sans insister. Ce cours te prépare surtout à l'autre rôle, celui que tu exerceras le plus souvent : écrire des programmes et des politiques de divulgation corrects, et traiter un signalement reçu avec sérieux et reconnaissance.

L'essentiel
La divulgation responsable = signalement privé, délai de correction raisonnable, publication après accord. Le bug bounty est un contrat : périmètre, règles, récompenses, non-poursuite dans les règles. Sans autorisation écrite, tester le système d'autrui est un délit — l'intention ne protège pas, seule l'autorisation le fait.

Exemple — Modèle de rapport de divulgation responsable (toute ressemblance est pédagogique)

```
# ---------- RAPPORT DE DIVULGATION (modèle) ----------
À : securite@exemple-organisation.invalid
Objet : Signalement responsable — contrôle d'accès sur les fiches membres
Programme : votre page « Sécurité » (périmètre : *.exemple-organisation.invalid)

Bonjour,
En utilisant mon propre compte de test dans le cadre de votre
programme de divulgation, j'ai observé le comportement suivant :

Résumé : la page de détail d'un membre semble ne pas vérifier
que le demandeur est bien le propriétaire de la fiche.
Attendu : un membre connecté ne devrait voir que sa propre fiche.
Observé : en modifiant l'identifiant dans l'adresse de MA fiche,
la fiche d'un autre membre de démonstration s'affiche.
Portée : je n'ai consulté aucune donnée réelle ; l'essai a été
fait entre mes deux comptes de test (n° T-101 et T-102).
Preuves : captures jointes, horodatées, identifiants de test seuls.
Impact : si confirmé sur des comptes réels, exposition de données
personnelles entre membres.
Suggestion : vérifier l'autorisation côté serveur sur chaque fiche.

Je reste disponible pour tout complément et je ne publierai rien
avant votre correction ou votre accord, conformément à vos règles.
# ---------- FIN DU MODÈLE ----------
```

À retenir

- Trois piliers : autorisation (périmètre écrit), démonstration minimale (ses propres comptes, pas de données d'autrui), signalement privé d'abord.
- Un programme de bug bounty est un contrat : qui sort du périmètre ou des règles sort aussi de la protection du programme.
- Recevoir un signalement est une chance : accuser réception, corriger, remercier — jamais menacer le découvreur de bonne foi.

Exercice

Un camarade te raconte, enthousiaste : « J'ai trouvé qu'en changeant un numéro dans l'adresse du site de la cantine, on voit les commandes des autres étudiants. Je vais tester sur tous les numéros ce soir pour prouver que c'est grave, puis je posterai les captures sur le groupe de la classe pour les faire réagir. » Liste tout ce qui cloche dans son plan au regard de cette leçon, et réécris son plan en version responsable, étape par étape.

✔ Corrigé

Ce qui cloche : (1) aucune autorisation — le site de la cantine n'a, à sa connaissance, aucun programme ; il teste donc le système d'autrui sans cadre. (2) « Tester sur tous les numéros » : il consulterait massivement les données personnelles d'autres étudiants, alors que la démonstration minimale (deux comptes à lui, ou un seul essai déjà constaté) suffit. (3) Publier des captures nominatives sur un groupe : divulgation publique avant correction, exposition de données d'autrui, et pression par la honte — tout l'inverse de la divulgation responsable. Version responsable : cesser les essais immédiatement ; noter sobrement ce qu'il a observé (date, adresse, comportement) sans refaire de tests ; signaler en privé au responsable du service ou à la direction informatique, par écrit, en proposant son aide pour vérifier la correction avec des comptes de test ; laisser un délai raisonnable ; ne rien publier de nominatif. Si le service ouvre ensuite un programme public, il pourra y participer dans les règles. Sa découverte initiale relève de l'usage normal du site ; c'est la suite prévue qui créait le problème — le plan responsable le transforme en découvreur utile au lieu d'en faire un fautif.

Leçon 7 — Projet : durcir la sécurité d'une application existante

Ce projet final applique tout le cours à une application réelle de l'équipe : BiblioIAM, notre bibliothèque, développée au fil des cours précédents sans préoccupation particulière pour la sécurité. La démarche professionnelle tient en quatre temps. D'abord, l'inventaire : quels composants, quelles versions, quelles données, quels comptes, quels secrets — et où vivent-ils ? Ensuite, l'état des lieux méthodique : passage de la grille de revue de code sur les modules sensibles, examen des journaux existants (ou de leur absence), lecture du registre des dépendances, vérification des configurations par défaut. Puis le plan de durcissement, priorisé par le risque et non par la facilité : on corrige d'abord ce qui expose le plus. Enfin, la validation : chaque correction est démontrée par un test d'abus qui échouait avant et qui échoue proprement — pardon : qui est rejeté — après.

Les chantiers types d'un tel durcissement se comptent sur une main et demie. Un : les secrets — tout sortir du code, rotation de ceux qui y ont vécu, variables d'environnement et modèle committé. Deux : les entrées et les requêtes — validation systématique, requêtes paramétrées partout, échappement à l'affichage. Trois : l'authentification et les autorisations — hachage adapté des mots de passe, vérification d'autorisation côté serveur sur chaque action, sessions qui expirent. Quatre : la configuration — HTTPS imposé, en-têtes de sécurité, messages d'erreur

génériques, comptes et mots de passe par défaut supprimés, services inutiles fermés. Cinq : la journalisation — événements de sécurité enregistrés et masqués, alertes branchées. Six : les dépendances — inventaire à jour, correctifs appliqués, surveillance activée. Chaque chantier est petit pris isolément ; c'est leur somme qui change le niveau réel de l'application.

Le projet se conclut par un rapport de durcissement : ce qui a été trouvé, ce qui a été corrigé, ce qui reste (et pourquoi, avec quelle mesure temporaire et quel responsable), et comment on a vérifié. Ce document est aussi important que les corrections : il servira à la prochaine équipe, à l'audit éventuel, et à toi-même dans six mois. Retiens enfin la leçon de méthode du cours entier : la sécurité n'est pas un chantier que l'on « termine » un vendredi soir, c'est un état que l'on entretient — revues à chaque fusion, secrets qui ne reviennent jamais dans le code, journaux relus, correctifs suivis. Le pipeline de la leçon 1 et l'automatisation vue en INF-L3-12 existent pour rendre cet entretien presque gratuit.

L'essentiel
Durcir = inventorier, examiner avec la grille, corriger par risque décroissant en six chantiers types (secrets, entrées/requêtes, authentification/autorisations, configuration, journaux, dépendances), valider chaque correction par un test d'abus, et rapporter ce qui reste. La sécurité est un entretien continu, pas un chantier unique.

Exemple — Plan de durcissement de BiblioIAM : extrait « avant / après » et en-têtes de sécurité

```
# ===== PLAN DE DURCISSEMENT — BiblioIAM (extrait) =====
# Chantier Avant (constat) Après (livré et testé)
# --------------- ------------------------------ ------------------------------
# 1. Secrets Clé de paiement dans Variable d'environnement,
# config.py, commitée clé ROTÉE, modèle .env.exemple
# 2. Requêtes 3 recherches par concaténation Requêtes paramétrées (100 %),
# test d'abus rejeté
# 3. Autorisation /admin vérifiait la connexion Rôle vérifié côté serveur
# seulement sur chaque action admin
# 4. Erreurs Traces techniques affichées Message générique affiché,
# détail journalisé (masqué)
# 5. Journaux Aucun journal de sécurité Événements + alertes A1/A2
# 6. Dépendances Versions inconnues Inventaire SBOM + correctifs

# ===== En-têtes de sécurité (configuration du serveur web) =====
# Strict-Transport-Security: max-age=31536000 (imposer HTTPS)
# Content-Security-Policy: default-src 'self' (limiter les sources)
# X-Content-Type-Options: nosniff (pas de devinette de type)
# Referrer-Policy: no-referrer (ne pas fuiter les adresses)
# X-Frame-Options: DENY (anti-incrustation de page)
#
# Reste à traiter (registre) : migration de l'ancien module d'images,
# responsable désigné, mesure temporaire en place, date de réexamen fixée.
```

À retenir

• Ordre de travail : inventorier, examiner, corriger par risque décroissant, valider, rapporter — jamais corriger au hasard.

• Six chantiers couvrent l'essentiel du durcissement d'une application web ordinaire.

• Le rapport de durcissement documente aussi ce qui n'est PAS corrigé : un reste assumé et daté vaut mieux qu'un reste caché.

Exercice

On te confie pour trois semaines le durcissement d'une application de gestion de notes utilisée par un lycée (données d'élèves mineurs, accès par les enseignants et la direction, hébergée sur un petit serveur). Tu ne pourras pas tout faire : construis un plan en trois vagues (semaine 1, 2, 3) en justifiant l'ordre par le risque, et indique pour chaque vague le test qui prouvera que la correction fonctionne.

✔ Corrigé

Vague 1 (l'exposition immédiate) : secrets et accès — sortir tout secret du code et faire tourner ceux qui y ont vécu (test : le dépôt et son historique ne contiennent plus aucun secret, l'ancien mot de passe de base est refusé) ; vérifier les autorisations côté serveur sur les notes (test d'abus : un enseignant connecté demande la fiche d'une classe qui n'est pas la sienne et se voit refuser) ; mots de passe hachés correctement (test : la base ne contient aucune empreinte faible type MD5). La sensibilité particulière — données de mineurs — justifie de commencer par qui peut voir quoi. Vague 2 (les failles du code) : requêtes paramétrées et validation des entrées sur les modules de saisie des notes et de recherche (tests d'abus : saisies malformées rejetées sans erreur technique affichée), messages d'erreur génériques, en-têtes de sécurité et HTTPS vérifiés. Vague 3 (la tenue dans le temps) : journalisation des événements de sécurité avec masquage et alertes simples (test : une série d'échecs de connexion déclenche l'alerte), inventaire des dépendances et correctifs (test : le scan ne remonte plus de vulnérabilité critique corrigible), registre des restes et rapport de durcissement remis à la direction. L'ordre suit le risque : d'abord fermer les portes visibles, ensuite assainir le code, enfin installer l'entretien — chaque vague se prouvant par des tests, pas par des intentions.

Travaux dirigés (TD)

TD 1 — Revue de code d'un module de connexion

Voici, résumées fidèlement, les lignes sensibles du module de connexion d'une application de gestion de coopérative : (1) la requête de recherche du compte est construite ainsi : « SELECT * FROM membres WHERE email = ' » + email_saisi + « ' » ; (2) le mot de passe saisi est comparé directement à la colonne mot_passe, stockée telle qu'elle a été écrite à l'inscription ; (3) en cas d'échec, le message affiché est « Mot de passe incorrect pour le compte awa@exemple.ml » ou « Aucun compte avec cet e-mail » selon le cas ; (4) après cinq échecs, rien ne se passe : on peut réessayer indéfiniment ; (5) la connexion réussie est notée dans un journal qui contient l'e-mail et le mot de passe « pour le support » ; (6) le rôle « trésorier » est vérifié seulement en masquant le bouton correspondant dans le menu. Travail : dresse la liste des défauts de sécurité, classe-les par famille (injection, authentification, autorisation, exposition

d'informations, journalisation), et propose pour chacun la correction précise.

✔ Corrigé du TD 1
Défaut 1 — Injection : requête par concaténation → requête paramétrée obligatoire ; famille injection. Défaut 2 — Authentification : les mots de passe sont stockés en clair (la comparaison directe le prouve) → stocker un hachage adapté (fonction de hachage lente avec sel, type Argon2 ou bcrypt), comparer les empreintes, et prévoir la migration des comptes existants. Défaut 3 — Exposition d'informations : deux messages différents révèlent si un compte existe → un seul message générique « identifiants incorrects », identique dans les deux cas. Défaut 4 — Authentification : aucune limitation des essais → compteur d'échecs, ralentissement progressif ou blocage temporaire, et alerte dans les journaux (règle A1 de la leçon 4). Défaut 5 — Journalisation : un mot de passe ne s'écrit jamais, même « pour le support » → journaliser l'événement et l'identifiant du compte, jamais le secret ; purger les anciens journaux fautifs. Défaut 6 — Autorisation : masquer un bouton n'est pas un contrôle → vérification du rôle côté serveur sur chaque action de trésorerie. Six défauts, cinq familles : cet exercice est la grille de la leçon 2 appliquée — c'est exactement pour cela qu'on la suit dans l'ordre, ligne après ligne, au lieu de « regarder si le code a l'air propre ».

TD 2 — Un secret poussé sur un dépôt public : l'incident complet

Vendredi 18 h 40, un membre de l'équipe BiblioIAM s'aperçoit qu'il a poussé, mardi, un commit contenant le fichier .env de production — avec le vrai mot de passe de la base et la vraie clé de l'API de paiement — sur le dépôt public du projet. Le commit a été suivi de cinq autres commits. Deux camarades ont déjà tiré la dernière version. Travail : écris le plan d'action complet, dans l'ordre, pour ce soir et pour lundi. Précise ce qu'il faut faire des cinq commits suivants, des clones des camarades, et ce qu'il faut vérifier chez les deux fournisseurs (base de données et API de paiement). Explique pourquoi « supprimer le fichier dans un nouveau commit » ne résout rien.

✔ Corrigé du TD 2
Ce soir, dans l'ordre de la leçon 3. Étape 1 — ROTATION IMMÉDIATE : changer le mot de passe de la base de production et révoquer/régénérer la clé d'API de paiement chez le fournisseur. Tant que cette étape n'est pas faite, tout le reste est secondaire : le secret est public et des robots ont pu le copier dès mardi. Étape 2 — AUDIT : consulter les journaux de la base (connexions depuis des adresses inconnues entre mardi et maintenant) et le tableau de bord du fournisseur de paiement (opérations inhabituelles) ; consigner ce qui est trouvé, même « rien » — c'est une preuve de diligence. Étape 3 — PURGE : réécrire l'historique pour en retirer le fichier (outil de réécriture d'historique) y compris dans les cinq commits suivants, republier l'historique nettoyé, et imposer aux deux camarades de re-cloner proprement — leurs clones contiennent le secret dans l'ancien historique, ils ne doivent pas re-pousser l'ancienne branche. Étape 4 — LUNDI, prévention : .gitignore vérifié (.env et variantes), .env.exemple seul versionné, détecteur de secrets en pre-commit et dans le pipeline, protection du dépôt si la plateforme le permet, incident inscrit au registre avec dates et actions. Pourquoi la suppression simple ne résout rien : Git conserve l'historique ; le fichier resterait lisible dans le commit de mardi pour quiconque clone le dépôt, sans compter les copies déjà faites (robots, caches, clones). La suppression nettoie l'écran, pas l'exposition — seule la rotation rend le secret copié inutilisable.

Travaux pratiques (TP)

TP 1 — Durcir BiblioIAM en une séance, grille en main

Par binôme, sur la copie de travail de BiblioIAM (environnement de TP, données fictives) : 1) Inventaire (30 min) : liste les composants et versions, les comptes existants, les endroits où vivent les secrets ; ouvre le registre de durcissement. 2) Secrets (30 min) : sors tous les secrets du code vers des variables d'environnement, committe le .env.exemple propre et le .gitignore complet ; si un secret de TP a été commité pendant la séance, applique la procédure de rotation du cours sur le service de test. 3) Code (1 h) : passe la grille de revue sur le module d'authentification et la recherche de livres ; corrige requêtes, échappement et vérifications d'autorisation ; démontre chaque correction par un cas d'abus qui est désormais rejeté. 4) Journaux (30 min) : branche la journalisation des connexions et des refus avec masquage, et écris la règle d'alerte « échecs de connexion » ; provoque-la avec un compte de test. 5) Dépendances et rapport (30 min) : produis l'inventaire des dépendances, applique les correctifs disponibles, remplis le registre (corrigé / reste daté) et rédige une page de rapport de durcissement.

✔ Attendus du TP 1
Attendus vérifiables par l'enseignant : (1) un registre ouvert avec inventaire complet — composants, versions, comptes, emplacements des secrets ; (2) aucun secret dans le code ni dans l'historique de la séance (recherche par motifs), variables d'environnement en place, .env.exemple sans aucune vraie valeur ; (3) requêtes toutes paramétrées sur les modules revus, saisies malformées rejetées sans message technique, action d'administration refusée à un compte non administrateur lors du test d'abus ; (4) journaux structurés sans mot de passe ni jeton complet, alerte effectivement déclenchée par la série d'échecs du compte de test ; (5) inventaire des dépendances produit, correctifs appliqués ou exceptions datées dans le registre, rapport d'une page qui distingue clairement « corrigé et prouvé » de « reste à traiter ». Barème indicatif : secrets 4 points, code 6 points, journaux 4 points, dépendances 3 points, qualité du rapport 3 points. Un binôme qui « finit » sans tests d'abus ni rapport n'a pas terminé : en sécurité, ce qui n'est pas démontré n'est pas fait.

Exercices corrigés supplémentaires

Exercice 1

Tri de journaux. Une application produit ces cinq lignes : (a) « 09:02 connexion réussie, compte u88 » ; (b) « 09:03 jeton de session complet : eyJ-exemple-fictif… (valeur notée en entier) » ; (c) « 09:05 paiement refusé, carte se terminant par 4417, référence TX-2091 » ; (d) « 09:07 le mot de passe de la base a été lu depuis le fichier de configuration » ; (e) « 09:11 accès refusé : u88 → /admin/export-complet ». Classe chaque ligne : acceptable telle quelle / acceptable après masquage ou reformulation / à ne jamais produire. Justifie en une phrase chacune.

✔ Corrigé
(a) Acceptable telle quelle : événement de sécurité utile, acteur identifié, aucun secret. (b) À ne jamais produire : un jeton de session complet dans un journal équivaut à donner la clé de la session à quiconque lit le journal ; au mieux, on écrirait « session renouvelée » sans valeur, ou un masquage en fin de jeton pour le diagnostic. (c) Acceptable après reformulation légère : la référence de transaction et les quatre derniers chiffres sont la forme admise ; il faudrait vérifier qu'aucun autre élément de la carte n'apparaît — telle quelle, la ligne respecte déjà cette forme. (d) À ne jamais produire : annoncer dans un journal qu'un secret a été lu n'apporte rien au diagnostic et documente l'architecture des secrets pour un lecteur malveillant ; et si la valeur avait suivi, ce serait la faute maximale. (e) Acceptable telle quelle, et précieuse : c'est l'événement qui alimente la détection des explorations de droits (règle A2). Le classement illustre le réflexe : utile + sans secret = garder ; utile mais sensible = masquer ; secret ou commentaire d'architecture = ne jamais produire.

Exercice 2

Priorisation de correctifs. Pour une application de réservation de salles utilisée par le personnel d'une entreprise, exposée uniquement sur le réseau interne : (A) vulnérabilité critique 9,5 dans la bibliothèque d'upload de fichiers, la fonction d'envoi de photos étant désactivée dans la configuration depuis toujours ; (B) vulnérabilité

moyenne 5,5 d'affichage sans échappement dans le champ « motif de la réservation », visible par tous les employés ; (C) vulnérabilité élevée 8,0 dans le composant d'authentification unique (SSO), avec correctif disponible, alors que des tentatives d'exploitation d'une faille similaire sont signalées dans la presse spécialisée. Propose l'ordre de traitement et les mesures d'attente pour chacun.

✔ Corrigé
Ordre proposé : C, puis B, puis A en risque documenté. Cas C : le composant d'authentification est la porte d'entrée de toutes les autres protections ; même en réseau interne, une compromission d'identité contourne tout le reste, et le contexte d'exploitation active aggrave l'urgence — action immédiate : appliquer le correctif de l'éditeur après test rapide, journaliser et surveiller les connexions inhabituelles entre-temps. Cas B : gravité moyenne, mais exposition réelle et quotidienne sur une fonction active (le champ motif est utilisé et affiché à tous) : à corriger dans les jours qui suivent, par l'échappement systématique à l'affichage ; mesure d'attente inutile si la correction est rapide, sinon filtrage temporaire du champ. Cas A : score le plus haut, exposition la plus faible — la fonction vulnérable est désactivée ; le risque n'est pas nul (une réactivation par erreur reste possible) : documenter une exception datée, vérifier que la désactivation est bien effective et le restera (test après chaque déploiement), planifier la mise à jour ou le retrait du composant. Encore une fois : sévérité × exposition × exploitation active — et chaque décision écrite.

Exercice 3

Premier message de divulgation. Tu découvres, en utilisant normalement le site d'une petite librairie en ligne de ta ville (aucun programme annoncé, aucun test intrusif : simple navigation avec ton compte), qu'une page de confirmation de commande affiche le nom et l'adresse d'un autre client — tu n'as rien modifié, la page est arrivée ainsi par le lien « voir ma commande ». Rédige le message que tu enverrais au libraire : objet, ton, faits, ce que tu n'as pas fait, ce que tu proposes. Une dizaine de lignes suffit ; la forme compte autant que le fond.

✔ Corrigé
Modèle attendu (forme) — Objet : « Signalement confidentiel — une page de commande affiche les données d'un autre client ». Corps : présentation brève (client du site) ; faits sobres : date et heure, parcours normal (« en cliquant sur Voir ma commande depuis mon compte »), ce qui s'est affiché (coordonnées d'un autre client, sans les recopier ni les joindre en capture lisible — ou capture masquée) ; ce qui n'a PAS été fait : aucune modification d'adresse, aucun autre essai, aucune copie des données, aucune publication ; risque énoncé calmement : si la page se comporte ainsi pour d'autres, des données personnelles de clients sont exposées ; proposition : disponibilité pour décrire le parcours à leur prestataire et pour vérifier la correction avec son propre compte ; demande : un accusé de réception ; engagement : confidentialité du signalement en attendant la correction. Ce qui ferait échouer l'exercice : un ton menaçant ou marchand (« payez-moi ou je publie » transforme le signalement en extorsion), la copie des données du client dans le message, de nouveaux essais « pour vérifier l'ampleur », ou une publication sur les réseaux sociaux. La divulgation responsable tient en trois mots : privé, minimal, patient.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Le « shift left » dans un cycle de développement sécurisé signifie…

   A) Tester la sécurité plus longtemps à la fin

   B) Déplacer les activités de sécurité le plus tôt possible dans le cycle

   C) Confier la sécurité à l'équipe la plus à gauche de l'open space

2. Lors d'une revue de code orientée sécurité, le premier réflexe est de…

   A) Vérifier l'orthographe des commentaires

   B) Compter les lignes de code

   C) Suivre le chemin des données, des entrées vers les zones sensibles

3. Un fichier .env.exemple versionné dans le dépôt doit contenir…

   A) Les noms des variables, avec des valeurs fictives étiquetées « exemple »

   B) Les vraies valeurs, mais seulement celles de l'environnement de test

   C) Les vraies valeurs chiffrées avec un mot de passe écrit dans le README

4. Un secret vient d'être poussé par erreur sur un dépôt. La toute première action est…

   A) Supprimer le fichier dans un nouveau commit

   B) Révoquer le secret chez le fournisseur et en générer un nouveau

   C) Passer le dépôt en privé

5. Lequel de ces journaux est correctement écrit ?

   A) Connexion échouée, mot de passe saisi : …

   B) Jeton complet de l'utilisateur : …

   C) Connexion réussie, compte u1042, horodatage, résultat

6. Pour décider de l'urgence d'un correctif, on croise le score CVSS avec…

   A) L'exposition réelle du composant et la sensibilité des données

   B) La date de sortie du composant

   C) La taille de l'équipe

7. Dans un programme de bug bounty, le « périmètre » désigne…

   A) Le budget maximal du programme

   B) Les systèmes et types de failles que l'organisation autorise à tester

   C) La liste des chercheurs inscrits

8. La divulgation responsable demande au découvreur d'une faille de…

   A) Publier immédiatement pour forcer la correction

   B) Exiger une récompense avant tout signalement

   C) Signaler en privé et laisser un délai raisonnable de correction avant toute publication

9. Masquer un bouton « Administration » dans le menu d'un utilisateur non administrateur…

A) Suffit comme contrôle d'accès

B) Ne protège rien si le serveur ne vérifie pas le rôle sur l'action elle-même

C) Est équivalent à un mot de passe

10. Une exception de correctif impossible à appliquer tout de suite doit être…

A) Oubliée discrètement pour ne pas inquiéter

B) Documentée avec mesure de réduction du risque, responsable et date de réexamen

C) Publiée sur le site pour prévenir les utilisateurs

Corrigés du QCM

1. Réponse B — Détecter et empêcher les failles dès les exigences et la conception coûte bien moins cher que les corriger après coup.

2. Réponse C — C'est en suivant chaque entrée jusqu'à la base de données, à l'affichage ou aux fichiers qu'on repère injections, échappements manquants et contrôles absents.

3. Réponse A — Le modèle sert à documenter les variables attendues ; toute vraie valeur — même « de test » — est un secret valide qu'il faut tenir hors du dépôt.

4. Réponse B — Le secret exposé doit être considéré comme public pour toujours : seule la rotation le rend inutilisable. Supprimer le fichier laisse le secret lisible dans l'historique, et un dépôt devenu privé n'efface pas les copies déjà faites.

5. Réponse C — Un bon journal d'événement contient l'acteur, l'action, l'horodatage et le résultat — jamais de mot de passe ni de jeton complet.

6. Réponse A — Un score élevé sur un composant hors ligne et une fonction désactivée peut attendre derrière un score moyen exploité activement sur une fonction exposée.

7. Réponse B — C'est le cœur du contrat : tester dans le périmètre et selon les règles place le chercheur sous la protection du programme ; en sortir retire cette protection.

8. Réponse C — Signalement privé, démonstration minimale, patience pendant la correction : c'est ce qui protège les utilisateurs et la position juridique du découvreur.

9. Réponse B — Côté client, tout peut être contourné en appelant l'adresse directement ; l'autorisation se vérifie côté serveur, sur chaque action sensible.

10. Réponse B — Ce qui ne peut pas être corrigé immédiatement se documente et se surveille ; le silence est la seule option inacceptable.

Résumé — les notions importantes

• La sécurité se construit dans le cycle de développement : une activité par phase, du shift left aux exigences jusqu'au suivi des correctifs en exploitation.

• La revue de code suit le chemin des données avec une grille fixe : validation des entrées, requêtes paramétrées, échappement des sorties, autorisations vérifiées côté serveur, erreurs sobres.

• Aucun secret dans le code : variables d'environnement, fichier local ignoré, modèle .env.exemple sans vraies valeurs, coffre dédié pour les systèmes importants.

• Secret commité = rotation immédiate chez le fournisseur, audit des accès, purge de l'historique, prévention de la récidive — dans cet ordre.

• Les journaux de sécurité sont structurés, horodatés, centralisés en ajout seul, et ne contiennent jamais mots de passe ni jetons complets : on masque.

• Les correctifs se priorisent en croisant sévérité CVSS, exposition réelle et exploitation active ; l'inventaire des dépendances (SBOM) est le préalable de tout le reste.

• Divulgation responsable et bug bounty sont des cadres d'autorisation : périmètre écrit, démonstration minimale, signalement privé — hors cadre, tester le système d'autrui est un délit.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — L'intégration de la sécurité dans les pipelines CI/CD : un guide DevSecOps

Chaîne : Cloud Native Community Group Lomé · Langue : Français · Niveau : Intermédiaire

Une session communautaire qui montre comment brancher les contrôles de sécurité dans un pipeline de livraison : analyse de code, recherche de secrets et de vulnérabilités, sans ralentir l'équipe. C'est la version automatisée du cycle sécurisé de cette leçon.

https://www.youtube.com/watch?v=_YaMnoHB5aI

Leçon 2 — Spring Security Java expliqué de A à Z — Le Cours Complet

Chaîne : lioncode · Langue : Français · Niveau : Intermédiaire

Un cours complet, en Java, sur ce qu'une revue de code doit vérifier côté serveur : authentification, autorisations, hachage des mots de passe et protections intégrées. Regarde-le en gardant la grille de revue de la leçon sous les yeux.

https://www.youtube.com/watch?v=nCLqewT2Z4E

Leçon 3 — TU CRÉES UN SAAS AVEC L'IA ? ATTENTION À CES 5 ERREURS !

Chaîne : Ibrahima Boiro · Langue : Français · Niveau : Débutant à intermédiaire

Cinq erreurs de sécurité très concrètes dans les jeunes applications : clés d'API exposées, secrets placés côté navigateur, permissions excessives, code accepté sans relecture. Un excellent résumé vivant de la leçon sur les secrets et de la revue de code.

https://www.youtube.com/watch?v=pQNF1VAjgIU

Leçon 6 — [PODCAST] Hacking éthique : où est la ligne ? Du bug bounty à la divulgation responsable

Chaîne : ESET France · Langue : Français · Niveau : Tous niveaux

Un échange avec un chasseur de failles sur la frontière exacte entre recherche légitime et délit : autorisation, périmètre, signalement. À écouter après la leçon 6 pour entendre ces notions racontées par un praticien.

https://www.youtube.com/watch?v=XuoGnCVIBPs

Leçon 6 — Faut-il commencer le bug bounty en 2026 ? (IA, argent, solitude)

Chaîne : Fransosiche · Langue : Français · Niveau : Tous niveaux

Un retour d'expérience lucide sur les programmes de bug bounty : ce que les règles autorisent, ce que les récompenses représentent vraiment, et la discipline qu'exige la divulgation responsable. Utile pour démystifier le sujet avant de s'y intéresser de près.

https://www.youtube.com/watch?v=aN1DahbPR4Q

Sources et références

• OWASP — Application Security Verification Standard (ASVS) et Software Assurance Maturity Model (SAMM) : référentiels libres et gratuits sur owasp.org, pour structurer exigences et maturité.

• OWASP — Top 10 et Cheat Sheet Series : les risques applicatifs majeurs et des fiches de prévention concrètes par sujet (requêtes paramétrées, journalisation, gestion des secrets), sur owasp.org.

• Microsoft — Security Development Lifecycle (SDL) : la démarche historique qui a popularisé les activités de sécurité par phase, documentation officielle Microsoft.

• NIST — Secure Software Development Framework (SSDF), publication SP 800-218 : cadre de pratiques de développement sécurisé, disponible sur le site du NIST (csrc.nist.gov).

• ANSSI — Guides de bonnes pratiques (développement sécurisé, journalisation des systèmes, gestion des correctifs), publiés sur cyber.gouv.fr.

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !